Rabta Programming Language

Phase 1: Flex + Bison tree-walking interpreter with compile-time checked conversations

Ansa Mubasher, Ahmed Yasin, Rana Muhammad Abdullah

Contents

Rabta Programming Language **1**

1. Dependencies and installation **2**

2. Building **2**

3. Running **3**
Examples . . . 3
Tests . . . 4

4. Language reference **4**
4.1 Lexical structure . . . 4
4.2 Types . . . 5
4.3 Program structure . . . 5
4.4 Functions . . . 5
4.5 Variables and arrays . . . 5
4.6 Expressions and precedence . . . 5
4.7 Control flow . . . 6
4.8 Output . . . 6
4.9 Protocols, channels and actors . . . 6

5. The semantic pass **7**
5.1 Type checking (semantic.c) . . . 7
5.2 Protocol checking (protocol.c) . . . 7
5.3 Other errors . . . 8

6. Implementation **9**

7. Files **9**

8. Grammar summary **10**

Rabta Programming Language

Course: Compiler Construction (7th semester) **Authors:** Ansa Mubasher, Ahmed Yasin, Rana Muhammad Abdullah **Phase 1:** interpreter built with Flex, Bison and C only (no LLVM, no bytecode)

Rabta is a small concurrent, message-passing language. A program is a set of independent actors (`banda`) that talk to each other only through typed channels (`nali`). The order, direction and type of every message on a channel is written down as a conversation protocol (`baatcheet`), and the compiler checks **before running** that every actor follows it. Programs that would deadlock or exchange the wrong data are rejected with a compile-time error.

```
// Protocol: who speaks, in what order, with what type
baatcheet Hisaab shuru
```

```
    client bolega i64       // n
    server bolega i64       // sum of 1..n
khatam

banda server(sira s : Hisaab.server) shuru
    rakho n = suno s        // receive n
    rakho total = 0
    ghoom [i : 1 -> n + 1] chal
        total = total + i
    khatam
    bolo s, total           // send the answer; protocol now complete
khatam

kaam main() shuru
    nali line : Hisaab              // creates line.client and line.server
    paida server(line.server)       // spawn the server actor
    bolo line.client, 100           // send n
    rakho ans = suno line.client    // receive the answer
    dikhao(ans)                     // prints 5050
    waapis 0
khatam
```

```
$ ./rabta examples/hisaab.rabta
5050
```

If `main` calls `suno line.client` before `bolo line.client, 100`, Rabta refuses to run the program:

```
$ ./rabta tests/client_suno_first.rabta
tests/client_suno_first.rabta:22: error: protocol violation: Hisaab.client must
speak first (expected bolo i64, found suno)
1 error(s) found; program not executed.
```

1. Dependencies and installation

Everything runs on Linux (tested on Ubuntu 24.04 under WSL 2).

Tool	Tested version	Purpose
gcc	13.3	C compiler (C11 + POSIX threads)
flex	2.6.4	lexer generator
bison	3.8.2	LALR(1) parser generator
make	4.3	build
pandoc + pdflatex	3.1	only to regenerate README.pdf

Install the build tools:

```
sudo apt update
sudo apt install build-essential flex bison make
```

Optional, only needed for `make pdf`:

```
sudo apt install pandoc texlive-latex-recommended texlive-latex-extra
```

2. Building

```
make            # builds ./rabta
make test       # builds, then runs all examples and negative tests
make clean      # removes every generated file
```

```
make pdf        # regenerates README.pdf from README.md
```

`make` runs exactly the pipeline required by the course:

```
bison -d -Wall rabta.y        # -> rabta.tab.c, rabta.tab.h
flex rabta.l                  # -> lex.yy.c
gcc -std=gnu11 -Wall -Wextra -O2 -g -c ...   # every .c file
gcc ... -o rabta *.o -lpthread
```

The grammar has **0 shift/reduce and 0 reduce/reduce conflicts** (Bison prints no warnings with `-Wall`), and all C files compile with **no warnings** under `-Wall -Wextra`.

3. Running

```
./rabta program.rabta            # check, then run
./rabta --check program.rabta    # lexer + parser + semantic pass only
./rabta --ast program.rabta      # print the annotated AST, then run
```

Exit status:

Status	Meaning
value of `waapis` in `main` (mod 256)	the program ran (all examples return 0)
1	lexical, syntax, type or protocol error: the program was **not** run
2	runtime error (division by zero, thali index out of bounds)
64 / 66	bad command line / file cannot be opened

Examples

File	What it shows	Output
`examples/hisaab.rabta`	the proposal example: client asks a server actor for sum 1..n	`5050`
`examples/factorial.rabta`	recursion with `kaam`/`bulao`/`waapis`	0! .. 10!, 20!
`examples/bubble_sort.rabta`	bubble sort on a `thali` passed by reference	sorted array
`examples/fibonacci.rabta`	`jab_tak` loop	first 20 Fibonacci numbers
`examples/if_else.rabta`	`agar` / `warna agar` / `warna`, logical operators, precedence	
`examples/ping_pong.rabta`	two actors playing ping-pong over one channel	
`examples/three_actors.rabta`	three worker actors summing ranges in parallel	`4501500`
`examples/pipeline.rabta`	a chain of three actors, each with two endpoints	`576`
`examples/sort_server.rabta`	sending a whole `thali` as a message	sorted array

Expected outputs are stored in `examples/expected/*.out` and checked by `make test`.

```
$ ./rabta examples/ping_pong.rabta
starting ping-pong
ping 1
    pong 1
ping 2
    pong 2
ping 3
    pong 3
pinger got the last pong: 3
```

Tests

`make test` runs `tests/run_tests.sh`, which prints PASS/FAIL for every program:

- every `examples/*.rabta` must exit with 0 and print exactly its `.out` file;
- every `tests/*.rabta` must be rejected. The first line of each test, `// EXPECT: <message>`, gives the text that must appear on stderr. Compile-time errors must exit with status 1 **and print nothing on stdout**, which shows the program was not executed. Runtime-error tests declare `// EXPECT_EXIT: 2`.

Test	Rejected because
`client_suno_first`	client receives before it sends (the proposal's example)
`server_bolo_first`	server sends before it receives
`wrong_message_type`	client sends `i64` where the protocol says `thali[3]`
`recv_wrong_type`	server receives an `i64` message into a `thali[4]`
`unfinished_conversation`	`main` returns before receiving the answer
`unfinished_actor`	actor ends before sending its answer
`use_after_finished`	endpoint used after all protocol steps are done
`use_after_handoff`	`main` uses an endpoint it gave to an actor with `paida`
`endpoint_never_used`	nobody ever plays the server role
`branch_mismatch`	only one branch of `agar`/`warna` sends
`protocol_in_loop`	conversation inside a loop
`wrong_endpoint_role`	actor needs `Hisaab.server`, got `Hisaab.client`
`type_mismatch`	`i64 + thali`
`undeclared_variable`	variable used before `rakho`
`wrong_arg_count`	`bulao` with too many arguments
`missing_return`	a `kaam` path without `waapis`
`syntax_error`	missing `)`
`division_by_zero`	runtime error (exit 2)
`index_out_of_bounds`	runtime error (exit 2)

4. Language reference

4.1 Lexical structure

- **Comments** start with `//` and run to the end of the line.
- **Identifiers:** letters, digits and `_`, not starting with a digit.
- **Integers:** decimal literals that fit in a signed 64-bit integer. Negative numbers are written with unary minus (`-5`).
- **Strings:** `"text"`, with escapes `\n`, `\t`, `\\`, `\"`. They are only allowed as arguments of `dikhao`.
- Whitespace, including newlines, is not significant. Statements need no separator.

Keywords

Keyword	Meaning	Keyword	Meaning
`shuru` / `khatam`	begin / end	`baatcheet`	protocol declaration
`agar` / `warna`	if / else	`bolega`	protocol step: "X speaks T"
`jab_tak ... chal`	while loop	`banda`	actor declaration
`ghoom [...] chal`	range loop	`sira`	endpoint parameter type
`rakho`	declare a variable	`nali`	create a channel
`thali`	bounded array	`paida`	spawn an actor
`kaam`	function definition	`bolo`	send a message
`bulao`	function call	`suno`	receive a message
`waapis`	return	`dikhao`	print
`i64`	64-bit integer type		

`client` and `server` are **not** keywords. They are role names that are checked by the semantic pass, so an actor may be called `server`, as in the proposal.

4.2 Types

Type	Description
`i64`	signed 64-bit integer. Booleans are `i64`: 0 is false, anything else is true
`thali[N]`	bounded array of N `i64` values (1 <= N <= 10,000,000), zero-initialised
channel	created by `nali ch : P`; it has the two endpoints `ch.client` and `ch.server`
endpoint	`sira e : P.client` or `sira e : P.server` (actor parameters only)

Channels and endpoints are not values. They cannot be stored in variables, compared or printed. They can only be used with `bolo`, `suno` and `paida`. This keeps every endpoint statically known, which the protocol checker needs.

4.3 Program structure

A program is a list of protocol, actor and function declarations in any order. Declarations may refer to each other (recursion and forward references are allowed). There are **no global variables**. Execution starts at `kaam main()`, which takes no parameters and must return an `i64` with `waapis`.

4.4 Functions

```
kaam name(param, param, ...) shuru
    statements
khatam
```

- A parameter is written `x` or `x : i64` (an `i64` passed by value), or `thali a` (an array passed **by reference**, of any size).
- A function always returns an `i64`. **Every path must end in `waapis expr`**: a function whose end is reachable is a compile-time error.
- Calls use `bulao`, either as an expression or as a statement:

```
rakho f = bulao factorial(10)
bulao sort(nums, 8)                 // result discarded
dikhao(bulao add(2, 3))
```

4.5 Variables and arrays

```
rakho x = 5 * 2                     // declare an i64 (initialiser required)
x = x + 1                           // assign

thali a[5]                          // 5 zeros
thali b[5] = [3, 1, 4]              // [3, 1, 4, 0, 0]  (at most N values)
b[0] = b[1] + 10                    // index with [expr], 0-based
```

- Indices are checked at runtime. An out-of-range index is a runtime error (exit 2).
- A whole `thali` cannot be assigned or used in arithmetic. It can be printed, passed to a `kaam` by reference, sent as a message, or received into.
- Names are block-scoped. Each `shuru ... khatam` body, loop body and branch is a new scope. Redeclaring a name in the same scope is an error, and shadowing an outer name is allowed.

4.6 Expressions and precedence

From lowest to highest precedence:

Level	Operators	Associativity
1	`||`	left
2	`&&`	left
3	`== !=`	non-associative
4	`< <= > >=`	non-associative
5	`+ -`	left
6	`* / %`	left
7	unary `-`, `!`	prefix

Operands are evaluated left to right. `&&` and `||` short-circuit and give 0 or 1, and comparisons give 0 or 1. Arithmetic wraps around on overflow (two's complement). `/` and `%` truncate toward zero, and dividing by zero is a runtime error.

4.7 Control flow

```
agar x > 0 shuru                    // if
    ...
khatam warna agar x == 0 shuru      // else if
    ...
khatam warna shuru                  // else
    ...
khatam

jab_tak i < 10 chal                 // while
    ...
khatam

ghoom [i : a -> b] chal             // for i = a, a+1, ..., b-1
    ...
khatam
```

The range loop evaluates `a` and `b` once. `b` is **exclusive**, and `i` is a new `i64` variable that is local to the loop.

4.8 Output

```
dikhao(expr_or_string, ...)
```

`dikhao` prints its arguments separated by one space, followed by a newline. `i64` values are printed in decimal, a `thali` as `[1, 2, 3]` and strings as they are. Each `dikhao` line is printed atomically, so lines from different actors never mix.

4.9 Protocols, channels and actors

Protocol. A protocol is a fixed, non-empty sequence of steps. Each step says which role speaks and the type of the message (`i64` or `thali[N]`):

```
baatcheet Tartib shuru
    client bolega thali[6]
    server bolega thali[6]
khatam
```

Channel. `nali ch : P` creates a channel for protocol `P` with the two endpoints `ch.client` and `ch.server`.

Actor. An actor is declared like a function, but with `banda`. It has no return value and may not contain `waapis`. Its parameters may be `i64` values or endpoints:

```
banda worker(id, sira job : Hissa.server) shuru ... khatam
```

Actors cannot take `thali` parameters, because actors share nothing. Send the array as a message instead.

Spawning. `paida name(args)` starts the actor on a new thread. `i64` arguments are copied. Endpoint arguments are **handed over**: the spawner may not use that endpoint again. An actor may pass its own endpoint on to another actor in the same way.

Messaging.

```
bolo ep, expr                    // send an i64 or a whole thali
rakho x = suno ep                // receive an i64 into a new variable
x = suno ep                      // ... into an existing variable
a[i] = suno ep                   // ... into an array element
thali t[6] = suno ep             // receive a thali[6] into a new array
suno ep                          // receive and discard
```

Here `ep` is `ch.client`, `ch.server` or a `sira` parameter. `suno` is only allowed as a whole initialiser or assigned value, or as a statement. That way the textual order of statements is exactly the order of protocol actions. Messages are copied (a `thali` is deep-copied), so no memory is ever shared.

Program end. When `main` returns, the interpreter waits for every actor, including actors started by other actors, to finish. Then it exits with `main`'s return value.

5. The semantic pass

The semantic pass runs after parsing and before anything is executed. If it reports any error, the program is **not run** and `rabta` exits with status 1. Each error has the form `file:line: error: message`.

5.1 Type checking (semantic.c)

- undeclared variables, channels, endpoints, functions, actors and protocols;
- duplicate declarations (top level, and within one scope);
- type mismatches in operators, conditions, indices, assignments, `rakho`, `waapis`, call arguments, `dikhao` and `bolo`;
- argument counts and kinds for `bulao` and `paida`, including the endpoint protocol and role (`expected sira Hisaab.server, found sira Hisaab.client`);
- return rules: every `kaam` path must end in `waapis` with an `i64`, `waapis` is not allowed in a `banda`, `main` must exist, take no parameters and not be called;
- `sira` parameters only on actors, and `thali` parameters only on functions;
- protocol roles must be `client`/`server`, and array sizes must be in range.

The pass also annotates the AST. Every variable gets a frame slot, every function and actor gets its frame size, and every name points to its declaration. The interpreter (and later the LLVM generator) use these annotations directly.

5.2 Protocol checking (protocol.c)

Each endpoint is a small state machine whose state is "how many protocol steps are done". The checker walks the body of `main`, of every `kaam` and of every `banda` in program order and simulates these state machines statically. One endpoint is tracked per `ch.client`/`ch.server` created by `nali`, and one per `sira` parameter.

Situation	Error
`suno` when this role must speak	`Hisaab.client must speak first (expected bolo i64, found suno)`
`bolo` when this role must listen	`Hisaab.server must listen first (expected suno i64, found bolo)`
(later steps)	`... must speak at step 3 (...) / ... must listen at step 2 (...)`
wrong type sent	`Tartib.client sends the wrong message type at step 1 (expected bolo thali[3], found bolo i64)`

Situation	Error
wrong type received	`Hisaab.server receives the wrong message type at step 1 (expected suno i64, found suno into thali[4])`
use after the last step	`Hisaab.client is used after the conversation finished (...)`
unfinished at `waapis` / end of scope	`conversation on line.client (Hisaab.client) is unfinished at waapis (next expected: suno i64, step 2 of 2)`
endpoint never used	`endpoint line.server (Hisaab.server) is never used in main`
used after `paida` handed it away	`line.server (Hisaab.server) was handed to banda 'server' with paida on line 16 and cannot be used here any more`
handed away after being used	`cannot hand line.client (Hisaab.client) to banda 'x' after it has been used here`
branches disagree	`the branches of agar leave line.client (Hisaab.client) in different states (agar: 1 step(s) done, warna: 0 step(s) done)`
`agar` without `warna` talks	`agar without warna changes the state of ... ; add a warna branch that does the same conversation`
conversation inside a loop	`line.client (Hisaab.client) is used inside the ghoom loop on line 17; loop bodies must be protocol-neutral`

Branches (`agar`/`warna`). Both branches start from the same endpoint states and must end in the same states. An `agar` without `warna` must not change any endpoint. A branch that ends in `waapis` does not have to match, because its endpoints were already checked at that `waapis`.

Loops: chosen rule. A protocol has a fixed number of steps, but a loop may run any number of times. So a `jab_tak`/`ghoom` body must be **protocol-neutral**: it may not use, or hand to `paida`, any endpoint declared outside the loop. A channel created *inside* the loop body is allowed if its conversation finishes within the same iteration:

```
ghoom [i : 1 -> 4] chal
    nali c : Q                      // fresh channel each iteration: OK
    paida doubler(c.server)
    bolo c.client, i
    rakho r = suno c.client
khatam
```

What this guarantees. Every endpoint performs exactly the steps of its protocol, in order, with the right types, and one side always has the opposite view of the other. So two actors talking over one channel can never wait for each other forever, and can never receive data of an unexpected type.

Limitation. As with basic session types, a cycle of waits across several different channels (A waits for B on channel 1 while B waits for A on channel 2) is not detected.

5.3 Other errors

```
file:4: syntax error, unexpected dikhao                (parser)
file:3: error: unexpected character '@'                (lexer)
file:6: runtime error: division by zero                (exit 2)
```

```
file:6: runtime error: index 5 is out of bounds for thali 'a' of size 5
```

6. Implementation

```
 .rabta source
      |
  rabta.l  (Flex)      tokens + line numbers (yylineno -> Bison locations)
      |
  rabta.y  (Bison)     LALR(1) grammar, actions build the AST (ast.c)
      |                the start rule's action drives the steps below
  semantic.c           symbol tables, type checking, slot allocation
  protocol.c           protocol state-machine simulation
      |                any error -> report, exit 1, never execute
  interpreter.c        tree-walking execution
  runtime.c            pthreads actors + bounded channel queues
```

Interpreter. Each call of a `kaam`, and each actor, gets a frame: an array of slots indexed by the slot numbers from the semantic pass. There are no globals, so actors share nothing.

Runtime. Each actor runs on its own POSIX thread with a 16 MiB stack. A channel holds two bounded FIFO queues (capacity 16), one per direction. Each queue is protected by a mutex, with condition variables `not_empty` (used by `suno`, which blocks until a message arrives) and `not_full` (used by `bolo`). After `main` returns, `rt_join_all()` joins every actor thread, including actors spawned while joining, so the program exits cleanly.

Checks done. `make test` passes 28/28. The actor examples were each run 300 times with identical output. Every example and test was also run under AddressSanitizer (no memory errors or leaks) and ThreadSanitizer (no data races).

Phase 2 (LLVM). The AST is already annotated with types, frame slots and resolved declarations. Each `kaam` becomes an `llvm::Function` whose slots become `alloca`s, a `thali` becomes an array accessed with GEP, and each `banda` becomes a function started by `rt_spawn`. `bolo`/`suno` become calls to `rt_send`/`rt_recv` in the same `runtime.c`.

7. Files

File	Description
`rabta.l`	Flex lexer: keywords, identifiers, integers, strings, operators, `//` comments, line tracking
`rabta.y`	Bison grammar (0 conflicts), AST-building actions, error reporting, `main()` and the driver that runs the semantic pass and the interpreter
`ast.h`, `ast.c`	AST node, list and type definitions, constructors, `--ast` pretty printer, cleanup
`semantic.h`, `semantic.c`	scoped symbol table, type checking, call/return rules, slot allocation, error reporting
`protocol.h`, `protocol.c`	protocol checker: endpoint state machines, branches, loops, hand-over
`interpreter.h`, `interpreter.c`	tree-walking interpreter: frames, expressions, statements, calls, actors, messages
`runtime.h`, `runtime.c`	actor threads, channels as bounded queues (mutex + condition variables), thread joining
`Makefile`	`all`, `test`, `clean`, `pdf` targets
`examples/*.rabta`	valid programs; `examples/expected/*.out` holds their expected output
`tests/*.rabta`	programs that must be rejected, each with its `// EXPECT:` message
`tests/run_tests.sh`	test runner used by `make test` (prints PASS/FAIL)
`docs/Rabta_Compiler_Proposal.pdf`	the original project proposal
`README.md`, `README.pdf`	this document
`.gitignore`	ignores generated files (`lex.yy.c`, `*.tab.*`, `*.o`, `rabta`, build folders)

8. Grammar summary

```
program     : { protocol | actor | function }
protocol    : "baatcheet" ID "shuru" step { step } "khatam"
step        : ID "bolega" ( "i64" | "thali" "[" INT "]" )
actor       : "banda" ID "(" [ params ] ")" block
function    : "kaam"  ID "(" [ params ] ")" block
params      : param { "," param }
param       : ID [ ":" "i64" ] | "thali" ID | "sira" ID ":" ID "." ID
block       : "shuru" { stmt } "khatam"

stmt        : "rakho" ID "=" ( expr | recv )
            | "thali" ID "[" INT "]" [ "=" ( "[" [ args ] "]" | recv ) ]
            | ID "=" ( expr | recv )
            | ID "[" expr "]" "=" ( expr | recv )
            | "agar" expr block [ "warna" ( block | "agar" expr block ... ) ]
            | "jab_tak" expr "chal" { stmt } "khatam"
            | "ghoom" "[" ID ":" expr "->" expr "]" "chal" { stmt } "khatam"
            | "waapis" expr
            | "dikhao" "(" ( expr | STRING ) { "," ( expr | STRING ) } ")"
            | "bulao" ID "(" [ args ] ")"
            | "nali" ID ":" ID
            | "paida" ID "(" [ args ] ")"
            | "bolo" endpoint "," expr
            | recv
recv        : "suno" endpoint
endpoint    : ID | ID "." ID
expr        : expr binop expr | "-" expr | "!" expr | primary
primary     : INT | ID | ID "[" expr "]" | ID "." ID | "(" expr ")"
            | "bulao" ID "(" [ args ] ")"
args        : expr { "," expr }
```